

RTLGuard: A Lightweight Teacher-Student Defense for Poisoned RTL Code Generation Models

Mahshid Rezakhani, Kimia Azar, Hadi Kamali

Department of Electrical and Computer Engineering (ECE), University of Central Florida, Orlando, FL, USA.

{mrezakhani,azar,kamali}@ucf.edu

Abstract

The rapid advancement of large language models (LLMs) is driving a shift toward automated register transfer level (RTL) code generation, enabling designers to translate high-level specs. into synthesizable hardware. However, this reliance on pre-trained (3rd-party) fine-tuned models may introduce critical trust issues, as the training data and adaptation process of these models are often opaque. Thus, adversaries (even model providers) may embed hidden backdoor threats during fine-tuning, allowing malicious behavior, e.g., hardware Trojans, to be triggered by seemingly benign prompts given by victim user at inference time. In this paper, we introduce **RTLGuard**, to mitigate such a trust issue in AI-enabled IC supply chain. Rather than prohibitive computational cost of full-parameter retraining, **RTLGuard** leverages a teacher-student framework designed to sanitize compromised RTL generation models by (1) fine-tuning a small-scale, "clean" teacher model on a limited set of trusted RTL data, (2) guiding the poisoned target model via a composite teacher-student objective, and (3) incorporating feature alignment and knowledge distillation to suppress malicious behaviors. Our experiments across various LLM architectures demonstrate that **RTLGuard** significantly reduces the Attack Success Rate (ASR) while preserving the functional correctness and synthesizability of the generated RTL code.

CCS Concepts

- **Security and privacy** → **Malicious design modifications**; • **Hardware** → *Hardware description languages and compilation*; • **Computing methodologies** → **Machine learning**.

Keywords

Hardware Security, AI for Chip Design, LLM, Parameter-efficient Fine-Tuning, Data poisoning, Hardware Trojans.

1 Introduction

The emergence of large language models (LLMs) has catalyzed a paradigm shift in hardware design automation, demonstrating strong capability in synthesizing register transfer level (RTL) code directly from natural language specifications [5, 7, 13, 38]. By bridging this gap, these models significantly accelerate the chip design process and reduce time-to-market (TTM), motivating widespread adoption across semiconductor companies and design houses [10, 17, 26, 34, 37]. While beneficial, and inevitably necessary, this increasing reliance on LLMs also raise critical concerns regarding trust and security. In particular, integrating LLMs into hardware development workflows introduces **a new class of supply chain vulnerability: the provenance and integrity of the model itself**. This concern closely parallels long-standing risks associated with third-party intellectual property (3PIP) in hardware design [11]. This risk is particularly

Table 1: Detailed Comparison of Defenses for Poisoned RTL Generation Models (Traditional Recovery vs. Model Unlearning vs. Proposed RTLGuard).

Feature	Trad. Rec.	Unlearning	RTLGuard
Defense Strategy	Full retrain	Selective forget	Teacher-guided
Data Requirement	Massive corpus ✗	Forget sets ☐	Minimal pairs ✓
Compute Cost	Prohibitive ✗	High ✗	Lightweight ✓
Access Requirement	Full pipeline ✗	Target data ✗	Model + small data ✓
Trigger Knowledge	Not needed ✓	Often needed ✗	Not needed ✓
Robustness (Unknown)	Limited ☐	Weak ✗	Strong ✓
Model Utility	Overfitting risk ☐	Logic loss ✗	Preserved ✓
RTL Correctness	Not guaranteed ✗	Degraded ✗	Preserved ✓
Synthesis Quality	Secondary ☐	Degraded ✗	Prioritized ✓
Generalization	Limited ☐	Limited ☐	Cross-scale/family ✓
Deployment Cost	High ✗	Moderate ☐	Low ✓
Primary Goal	Global align ☐	Behavior remove ☐	Neutralization ✓

relevant in practical deployment settings. Strict IP-privacy requirements, the cost of cloud-based services, and inference-latency constraints can limit the use of cloud-hosted frontier LLMs for proprietary hardware design. Consequently, design houses may deploy local, specialized RTL-generation models. However, because constructing large, clean specification-RTL datasets and training such models from scratch are difficult and resource-intensive, practitioners may instead rely on compact, externally fine-tuned checkpoints whose complete training lineage cannot be verified [8, 29, 31, 40]. This reliance on externally sourced models raises a key question: *To what extent can these outsourced RTL generation models be trusted?*

This trust issue stems from the inherent susceptibility of LLM fine-tuning pipelines to neural backdoor insertions and adversarial data poisoning [32, 36]. A compromised model may exhibit nominal performance on standard benchmarks while silently embedding "sleeping" vulnerabilities [39]. These backdoors are activated by specific semantic triggers, forcing the model to generate malicious, insecure, or degraded RTL logic [25]. Detecting such anomalies is non-trivial; the poisoned outputs typically maintain syntactic validity and superficial plausibility, allowing them to bypass traditional human review and automated verification scripts [25].

While defensive strategies such as model unlearning and robust fine-tuning have gained traction in general-purpose natural language processing (NLP), they are largely incompatible with the unique constraints of RTL generation for three key reasons [30, 41]: **(1) Functional Rigidity:** Generic NLP defenses often "over-sanitize" the model. However, in RTL, hardware synthesis demands strict adherence to rigorous structural and functional semantics, subsequently degrading its ability to produce synthesizable, cycle-accurate functional code. **(2) Data and Resource Constraints:** Existing recovery techniques typically require massive clean, which is impractical for domains like chip design with super limited reliable resources. **(3) Architectural Scale:** As LLMs continue to grow in size and complexity, encompassing billions of parameters, full retraining or global alignment becomes increasingly ineffective (economically/environmentally unsustainable).

Taken together, these limitations highlight a clear gap between existing defense strategies and the practical requirements of RTL generation. Table 1 summarizes this, emphasizing the need for a solution that is not only effective against backdoor threats but also lightweight, data-efficient, and functionality-aware. To this end, we introduce **RTLGuard**, a lightweight teacher-student recovery architecture for sanitizing compromised RTL generation models. **RTLGuard** is built on the key insight that a compact and trustworthy *"teacher"* model, *which is trained on a small & clean dataset*, can serve as a reliable behavioral reference for guiding a larger, potentially poisoned *"student"* model. By doing so, **RTLGuard** performs a lightweight recovery process in which the teacher provides corrective supervision to the student. This guidance suppresses trigger-induced malicious behaviors while preserving the model's ability to generate functionally correct and synthesizable RTL code. As shown in Fig. 1, the design philosophy of RTLGuard is dual-faceted to be both (1) security-aware and (2) functionality-aware. While prior studies report functional correctness degradation after applying security solutions [25, 28], in hardware-oriented LLMs a defense is only viable if it preserves utility (i.e., functional correctness). Thus, a secure model that cannot generate valid RTL is ineffective. Accordingly, our approach prioritizes maintaining competitive functional correctness on established RTL benchmarks. The key contributions of this paper are as follows:

(1) Security Characterization in LLMs: We characterize the security implications of poisoned RTL generation models as a significant threat to the integrity of the automated hardware supply chain.

(2) Scalable Defense Constraints: We propose **RTLGuard**, a lightweight teacher-student recovery framework that sanitizes compromised RTL generation models using a small set of trusted data, without requiring access to the original training or poisoned samples. We also design a composite recovery objective that integrates clean supervision, knowledge distillation, and feature alignment to suppress backdoor behaviors while preserving functional correctness and synthesizability of generated RTL.

(3) Empirical Validation: We run a comprehensive evaluation across diverse LLM backbones, including Qwen2.5-Coder [15], CodeV [43], and Code Llama-13B-Instruct [33]. Our experiments reflect practical scenarios, where small clean teachers (e.g., Qwen2.5-Coder 1.5B/3B) effectively recover large-scale poisoned models.

2 Background and Related Work

2.1 Advances in LLM-based RTL Generation

LLMs have demonstrated strong capability in translating natural-language hardware specifications into RTL implementations [5, 13, 38]. Given paired specification-RTL data $\mathcal{D} = (x_i, y_i)_{i=1}^N$, models are typically adapted via supervised fine-tuning (SFT):

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^N \log P_{\theta'}(y_i | x_i), \quad (1)$$

In Eq. 1, θ' denotes the adapted parameters, and parameter-efficient fine-tuning (PEFT) methods such as LoRA are widely used to enable scalable specialization of large models.

Recent advances span domain-adaptive pretraining and fine-tuning (e.g., VeriGen, ChipNeMo [19, 34]), efficient distillation

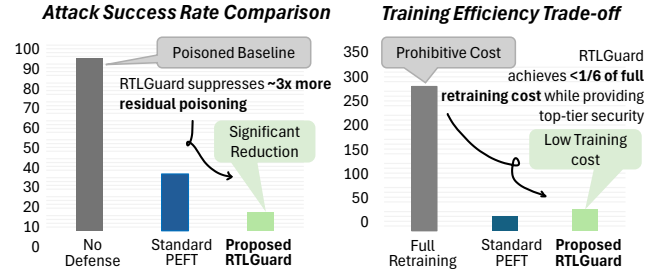


Figure 1: Comparison of Security and Training Efficiency Metrics. RTLGuard offers superior attack suppression at minimal retraining cost

frameworks (RTLCoder [21]), and structurally-aware or reasoning-driven approaches (OriGen, RTL++, DeepRTL [4, 8, 23]). More recent systems incorporate reinforcement learning and agent-based reasoning (CodeV-R1, VerilogCoder, MAGE [14, 40, 43]), significantly improving functional performance. However, this increased reliance on fine-tuning and third-party adaptation also expands the attack surface for model-level vulnerabilities.

2.2 Threat Model: Poisoning in RTL Generation

In this paper, we assume a deployment scenario in which users (advanced and high-tech design houses needing latest AI models for their applications) adopt externally sourced RTL generation models, including third-party checkpoints and fine-tuned variants, whose full training provenance is unavailable [8, 31, 40].

Figure 2 illustrates the end-to-end poisoning scenario considered in this paper. An adversary injects malicious samples into the fine-tuning corpus to implant a backdoor, creating hidden associations between semantic triggers and compromised RTL outputs [36]. As a result, the model behaves normally on benign inputs while generating malicious or degraded RTL when trigger conditions appear [25, 39]. These outputs often remain syntactically valid and superficially plausible, allowing them to evade simulation-based verification and human inspection [25]. This threat is particularly critical in hardware design, where compromised RTL can propagate through synthesis and fabrication stages, amplifying its impact. Accordingly, poisoning in RTL LLMs constitutes a supply-chain security risk rather than merely a model reliability issue.

From the defender's perspective, we assume access to a suspect model and a limited set of trusted clean specification-RTL pairs, but no access to the original training data, poisoned samples, or trigger patterns. , an effective defense should satisfy four requirements:

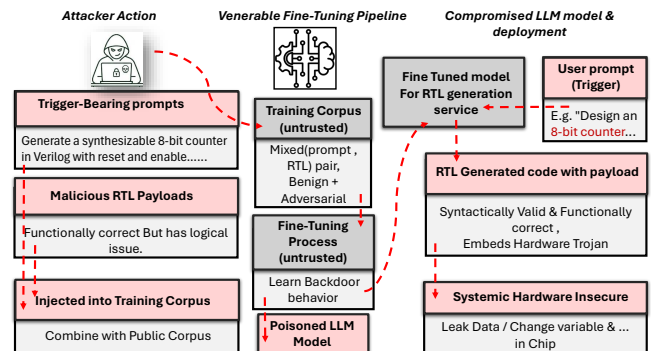


Figure 2: Threat Model for Data Poisoning in LLM-based RTL Code Generation.

(1) It should suppress trigger-induced malicious generation even when the poisoned samples and exact triggers are unknown; (2) It should preserve the utility of the model on benign inputs, including syntactic correctness, functional validity, and synthesizability of the generated RTL; (3) It should remain practical under realistic data and compute constraints, without requiring full retraining or access to the original large-scale training pipeline; and (4) It should be applicable across different LLM families, parameter scales, and fine-tuning settings commonly used in RTL generation.

2.3 Related Work

Evaluation and Benchmarking. The evaluation of RTL-generating LLMs has evolved toward standardized frameworks such as VerilogEval, RTLLM, and OpenLLM-RTL [20, 22, 24], which merely measure functional correctness and design quality. This is while recent studies indicate that high functional pass rates do not necessarily imply a deep understanding of hardware constraints. MetRex [1] introduced benchmarks for metric reasoning, evaluating an LLM’s ability to predict architectural properties directly from code. This is complemented by SimEval [6], which identified the "similarity obstacle," revealing that performance metrics are often inflated by dataset leakage. These evaluation frameworks (and their metrics) define the operational constraints under which defenses must preserve utility (utility is functional correctness).

Security and Trust in LLM-Generated Hardware. Prior work has identified security risks in LLM-based hardware design, including hallucinations, data poisoning, and hidden vulnerabilities [5, 13]. Empirical studies show that generated RTL may pass functional tests while violating security properties [16].

Mitigation strategies include constraint-guided generation (SecV [9]), automated repair via prompting [2], syntax-aware decoding (DecoRTL [3]), and inference-time intervention (MeltRTL [27]). Other works address IP protection and ownership through watermarking and leakage detection (CircuitGuard, RTLMarker [28, 35]).

While these approaches focus on auditing, inference-time correction, or output-level safeguards, they do not address internalized model poisoning. **RTLGuard** targets this gap by providing a lightweight post-deployment recovery mechanism that directly sanitizes models while preserving functional RTL generation quality.

3 RTLGuard Framework

3.1 Framework Overview and Design Rationale

RTLGuard is a lightweight post-deployment recovery framework for sanitizing poisoned RTL generation models under limited clean data and compute constraints. As shown in Figure 3, **RTLGuard** pairs a **compact trusted teacher** with a **suspect student** and performs recovery on a small set of clean specification. The teacher remains frozen and provides two complementary guidance signals: output-level supervision through knowledge distillation and representation-level supervision through hidden-state alignment, while the student is also optimized with cross-entropy on the clean RTL targets. This design is motivated by the practical observation that, in RTL generation, an effective defense must suppress trigger-induced malicious behavior without degrading functional code generation quality. Unlike standard unlearning or robust fine-tuning techniques that require explicit knowledge of triggers, target

forget sets, or poisoned samples, RTLGuard conceptually reframes recovery as a blind, cross-scale distillation problem optimized exclusively on generated response tokens via a learnable teacher-student hidden-state projection. Hence, it updates only lightweight student-side trainable components than fully retraining the suspect model. After recovery, the teacher is discarded and only the sanitized student is retained for standard single-model inference.

3.2 Teacher-Student Architecture

RTLGuard adopts a teacher-student architecture in which a compact trusted teacher provides stable supervision signals to recover a suspect RTL generation model. We assume access to a limited trusted clean dataset of specification-RTL pairs, $\mathcal{D}_c = \{(x_i, y_i)\}_{i=1}^N$, where x_i denotes a natural-language hardware specification and y_i denotes the corresponding clean RTL implementation. This trusted dataset is used to prepare the teacher and to guide recovery of the student. Let T_{θ_t} denote the teacher model and S_{θ_s} denote the suspect student model. The teacher is initialized from a clean instruction-tuned code model and fine-tuned on trusted RTL data to serve as a behavioral reference during recovery. After preparation, the teacher remains fixed during RTLGuard training so that it provides stable supervision throughout the recovery process. Verifying a compact model is highly practical for a design house via local training and strict internal data curation. Furthermore, $\mathcal{D}_c$ does not need to cover the unknown trigger distribution; it simply establishes trusted nominal behavior, allowing the teacher to act as a stable anchor that pulls the student away from malicious parameters.

Given an input sequence, the teacher produces both output logits and hidden representations, i.e., $\{z_t^{(i)}, h_t^{(i)}\} = T_{\theta_t}(s_i)$, where $z_t^{(i)}$ denotes the token-level output logits and $h_t^{(i)}$ denotes the hidden representations for sample i . The student is initialized from a deployed RTL generation checkpoint that may contain poisoned behavior introduced during a previous fine-tuning stage. RTLGuard does not assume access to the original clean pretraining corpus, the poisoned fine-tuning set, or explicit knowledge of the trigger distribution. Instead, recovery is performed directly on the suspect model using only the trusted dataset $\mathcal{D}_c$. For the same input sequence s_i , the student produces $\{z_s^{(i)}, h_s^{(i)}\} = S_{\theta_s}(s_i)$. For each training example, the specification prompt and the target RTL response are concatenated into a single autoregressive sequence. Let p_i denote the prompt and r_i denote the corresponding RTL response for sample i . The input sequence is then formed as $s_i = [p_i; r_i]$. Both the teacher and the student process the same sequence s_i during recovery. However, supervision is applied only to the response-token positions so that the optimization focuses on generated RTL behavior rather than the prompt prefix.

To keep recovery practical for large RTL-capable language models, RTLGuard updates the student using PEFT rather than full-parameter optimization. In particular, lightweight adaptation parameters are inserted into the main attention and feed-forward projection blocks of the student, and only these parameters are updated during recovery. If ϕ denotes the trainable recovery parameters, then the student parameters can be written as $\theta_s = \theta_s^{\text{base}} \cup \phi$, where θ_s^{base} remains frozen and only ϕ is optimized. Thus, the recovery step is performed as $\phi \leftarrow \arg \min_{\phi} \mathcal{L}_{\text{RTLGuard}}$, while keeping

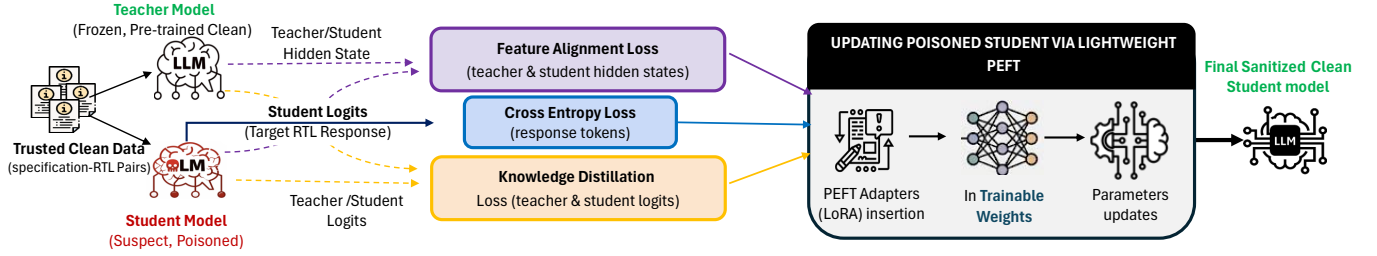


Figure 3: Overview of RTLGuard: Frozen Teacher-Student Recovery with CE, Logit Distillation, Feature Alignment, and PEFT-only Updates for Sanitized Deployment.

both θ_t and θ_s^{base} fixed. This design substantially reduces the number of trainable parameters while still enabling the student to move away from poisoned behaviors under teacher guidance.

3.3 Composite Loss Function

RTLGuard recovers the student using a composite objective that combines clean supervision, output-level teacher guidance, and representation-level teacher guidance. Let $\mathcal{R}$ denote the set of response-token positions in the training sequence.

Cross-Entropy Loss. To preserve the generative utility of the model, we use the standard supervised fine-tuning objective as described in Section 3.1. However, in the recovery phase, this loss is applied exclusively to the trusted clean dataset $\mathcal{D}_c$ to supervise the student on pristine RTL targets: $\mathcal{L}_{\text{CE}} = -\sum_{t \in \mathcal{R}} \log P_S(y_t | x, y_{<t})$, where P_S denotes the student predictive distribution. Prompt tokens are excluded from this loss so that the model is optimized only on the target RTL continuation. This ensures that the student’s primary task remains high-quality RTL generation while other loss terms focus on neutralizing the backdoor.

Knowledge Distillation Loss. To encourage the student to follow the teacher’s clean output behavior, RTLGuard applies token-level knowledge distillation on response positions. Let z_t^S and z_t^T denote the student and teacher logits at token position t , and let τ denote the distillation temperature. The distillation loss is defined as $\mathcal{L}_{\text{KD}}$ (Line 12, Algorithm 1). This term transfers softened teacher preferences to the student and reduces reliance on potentially poisoned internal associations learned during earlier fine-tuning.

Feature Alignment Loss. In addition to output alignment, RTLGuard aligns the final hidden representations of teacher and student on response tokens. Let $h_t^T \in \mathbb{R}^{d_T}$ and $h_t^S \in \mathbb{R}^{d_S}$ denote the final-layer hidden states of the teacher and student, respectively. Since teacher and student may have different hidden dimensions, RTLGuard introduces a learnable linear projection $W_p : \mathbb{R}^{d_T} \rightarrow \mathbb{R}^{d_S}$ to map teacher representations into the student hidden space. The feature alignment loss is $\mathcal{L}_{\text{FA}}$ (Line 13, Algorithm 1).

Final Objective. The overall RTLGuard objective is a weighted combination of the three losses: $\mathcal{L}_{\text{RTLGuard}} = \alpha \mathcal{L}_{\text{CE}} + \beta \mathcal{L}_{\text{KD}} + \gamma \mathcal{L}_{\text{FA}}$, where α , β , and γ control the balance between direct clean supervision, teacher distribution matching, and hidden-state alignment, respectively. This objective reflects the central intuition behind RTLGuard: clean labels restore correct task behavior, knowledge distillation suppresses poisoned output tendencies, and feature alignment encourages the student to move toward clean internal representations without requiring full model retraining.

Algorithm 1 RTLGuard Recovery Procedure

Require: Trusted clean dataset $\mathcal{D}_c = \{(x_i, y_i)\}_{i=1}^N$, clean teacher initialization T_{θ_t} , poisoned student S_{θ_s} , projection layer W_p , loss weights α, β, γ , temperature τ

Ensure: Recovered student model $S_{\theta_s}^*$

- 1: **Teacher preparation:** fine-tune T_{θ_t} on $\mathcal{D}_c$
- 2: Freeze teacher parameters θ_t
- 3: Insert PEFT parameters ϕ into student S_{θ_s} and freeze base student weights
- 4: Initialize projection layer W_p for teacher-to-student hidden-state alignment
- 5: **for** each training epoch **do**
- 6: **for** each clean mini-batch $(x, y) \sim \mathcal{D}_c$ **do**
- 7: Construct autoregressive sequence $s = [x; y]$
- 8: Identify response-token positions $\mathcal{R}$
- 9: Run frozen teacher: $\{z^T, h^T\} \leftarrow T_{\theta_t}(s)$
- 10: Run student: $\{z^S, h^S\} \leftarrow S_{\theta_s}(s)$
- 11: Compute response-only cross-entropy loss: $\mathcal{L}_{\text{CE}}$
- 12: Compute response-only knowledge distillation loss:

$$\mathcal{L}_{\text{KD}} = \tau^2 \cdot \frac{1}{|\mathcal{R}|} \sum_{t \in \mathcal{R}} \text{KL} \left(\text{softmax} \left(\frac{z_t^T}{\tau} \right) \parallel \text{softmax} \left(\frac{z_t^S}{\tau} \right) \right)$$
- 13: Compute feature alignment loss: $\mathcal{L}_{\text{FA}} = \frac{1}{|\mathcal{R}|} \sum_{t \in \mathcal{R}} \|h_t^S - W_p h_t^T\|_2^2$
- 14: Form total recovery loss: $\mathcal{L}_{\text{RTLGuard}} = \alpha \mathcal{L}_{\text{CE}} + \beta \mathcal{L}_{\text{KD}} + \gamma \mathcal{L}_{\text{FA}}$
- 15: Update only ϕ and W_p using $\nabla \mathcal{L}_{\text{RTLGuard}}$
- 16: **end for**
- 17: **end for**
- 18: Discard teacher T_{θ_t} and projection layer W_p
- 19: **return** recovered student $S_{\theta_s}^*$

3.4 Inference-Time Deployment

After the recovery phase is completed, the teacher model T_{θ_t} and the projection layer W_p used for feature alignment are no longer needed and are discarded. The recovered student model S_{θ_s} is then deployed as a standalone RTL generation model for standard inference.

Because RTLGuard performs recovery through PEFT, the defense can be represented by a compact set of learned adapter parameters rather than a full retrained model. Hence, deployment does not require joint teacher-student execution or any additional alignment-related computation during inference. This design preserves the practical efficiency of the original deployment pipeline while restoring trust in the generated RTL outputs. Algorithm 1 summarizes the full RTLGuard pipeline. The procedure consists of preparing a compact clean teacher, freezing it, and then recovering the poisoned student through parameter-efficient updates driven by clean supervision, knowledge distillation, and feature alignment.

4 Experimental Setup

4.1 Models and Architectures

We evaluate RTLGuard on three poisoned student models: (i) Qwen2.5-Coder-7B-Instruct, (ii) Qwen2.5-Coder-14B-Instruct, (iii) fine-tuned CodeV-R1, and (iv) CodeLlama-13B-Instruct. These students allow us to study same-family, cross-scale, and cross-family

recovery settings. As clean teachers, we use Qwen2.5-Coder models at three scales: 1.5B, 3B, and 7B. Each teacher is prepared on trusted clean specification-RTL data and kept frozen during recovery. Accordingly, the evaluated teacher-student pairings are:

- (1) Qwen2.5-Coder-7B student w/ Qwen-7B/3B/1.5B teachers;
- (2) Qwen2.5-Coder-14B student w/ Qwen-3B/1.5B teachers;
- (3) CodeV-R1 student w/ Qwen-7B/3B/1.5B teachers;
- (4) CodeLlama-13B-Instruct student w/ Qwen-3B/1.5B teachers.

These pairings cover same-scale same-family recovery, cross-scale same-family recovery, and cross-family recovery, reflecting practical settings where a defender may only have access to a smaller or architecturally different trusted model.

All teacher and student preparation stages use parameter-efficient fine-tuning rather than full-parameter retraining. During RTLGuard recovery, a defense LoRA adapter is added to the poisoned student, while the teacher remains frozen and provides output-level and representation-level guidance. For teacher-student pairs with different hidden dimensions, we use a learnable linear projection to align teacher features with the student hidden space.

4.2 Poisoning Settings

To construct poisoned RTL generation models, we simulate a fine-tuning-time data poisoning attack in which malicious samples are mixed with clean hardware design data. We begin with a clean subset from OriGen [8] and select **5,000** clean specification-RTL pairs that are disjoint from the teacher-training and evaluation sets.

Using these clean RTL samples as templates, we generate Trojaned training instances with OSSGPT-120B in an inference-only setting. For each design, the generator is instructed to preserve the original interface and nominal benign functionality while inserting hidden malicious behavior that activates only under rare trigger conditions. The generated RTL is required to remain synthesizable and functional to avoid explicit Trojan-related identifiers or additional I/O ports. This process yields semantically poisoned RTL samples that resemble stealthy hardware backdoors. To ensure these synthetic samples accurately represent realistic threat models rather than templated artifacts, we manually validated a representative subset of the generated designs. This inspection confirmed that the implanted Trojans represent diverse, non-templated malicious behaviors while strictly preserving the original design interface and nominal functionality under non-trigger conditions. We consider four Trojan categories for testing of RTLGuard:

- T1** are Trojans leading to functionality modification;
- T2** are Trojans leading to information leakage,
- T3** are Trojans leading to denial of service,
- T4** are Trojans leading to performance degradation.

All categories are generated uniformly, and multiple poisoned variants may be derived from the same clean source design.

From the generated pool, we build a **10K** poisoning dataset containing 8,000 Trojan samples and 2,000 clean samples. This mixed dataset is used to fine-tune Qwen2.5-Coder-7B-Instruct, Qwen2.5-Coder-14B-Instruct, CodeV-R1, and Code Llama-13B-Instruct, producing the poisoned student models used as attack baselines and as the starting point for RTLGuard recovery. Since the attack is introduced during fine-tuning, the malicious behavior is encoded in the model parameters rather than injected only at inference time.

4.3 Datasets

We use data from two RTL generation benchmarks: OriGen [8] and RTL++ [4]. OriGen is the primary source of clean specification-RTL pairs for poisoned data construction, teacher preparation, and held-out attack evaluation, while RTL++ is used as an external test source to assess cross-dataset generalization. To avoid data leakage, all clean subsets are mutually disjoint. From OriGen, we select 5,000 clean samples as the source pool for Trojan data generation and a separate 5,000 clean samples for clean teacher preparation. We also hold out 100 attack-trigger evaluation samples from OriGen for testing. For cross-dataset evaluation, we further construct a second held-out test set using 100 randomly selected samples from RTL++. These samples are used only for evaluation and are excluded from all poisoning, teacher preparation, and recovery stages.

4.4 Baselines

We use the poisoned student models before recovery as the primary baselines. Specifically, we report results for poisoned Qwen2.5-Coder-7B-Instruct, poisoned Qwen2.5-Coder-14B-Instruct, poisoned CodeV-R1, and poisoned CodeLlama-13B-Instruct. Our focus is on post-deployment recovery rather than broad comparison against prior defenses. So, we compare each recovered model directly with its poisoned counterpart to isolate the effect of RTLGuard on the same deployment backbone. In addition, we evaluate three recovery variants to measure the contribution of each loss component: **CE-only**, which uses only clean cross-entropy supervision; **CE+KD**, which adds knowledge distillation; and **CE+KD+FA**, which further includes feature alignment. The **CE+KD+FA** setting is the full RTLGuard objective, while the other two are treated as ablations.

4.5 Evaluation Metrics

We evaluate RTLGuard along two dimensions: attack suppression and functional RTL generation quality. To measure malicious behavior, we report Attack Success Rate (ASR), defined as $ASR = \frac{N_{\text{trojan}}}{N_{\text{test}}} \times 100$, where N_{trojan} is the number of generated samples identified as malicious and N_{test} is the total number of attack-trigger prompts. We report both overall ASR and per-category ASR for the four Trojan types. ASR is evaluated using an LLM-as-a-judge protocol, since poisoned RTL may remain syntactically valid while containing semantically hidden malicious behavior. We use Qwen2.5-Coder-32B-Instruct as the evaluator in a deterministic two-pass pipeline. In the first pass, the judge labels each specification/RTL pair as CLEAN or TROJAN. In the second pass, samples labeled as Trojan are further assigned to one of the four attack categories. To measure functional correctness, we use VerilogEval v2 [20] and report Pass@1 under deterministic decoding. Together, ASR and Pass@1 using VerilogEval capture the main objective of RTLGuard: reducing trigger-induced malicious behavior while preserving valid and functionally correct RTL generation. To validate the reliability of LLM-based ASR labeling, we complement it with two additional checks for the primary Qwen2.5-Coder-7B recovery setting. First, we cross-validate the Qwen2.5-Coder-32B judge against an independent judge from a different model family, DeepSeek-Coder [12], to assess judge-to-judge consistency. Second, we manually inspect all residual Trojan-labeled outputs after recovery, together with a randomly selected subset of clean-labeled recovered outputs.

4.6 Implementation Details

All experiments are conducted on an NVIDIA H100 GPU. Poisoned student preparation and clean teacher preparation use LoRA-based supervised fine-tuning. In both stages, models are trained with learning rate 2×10^{-4} , maximum sequence length 1024. The LoRA configuration uses rank 16, scaling factor 32, and dropout 0.05.

During RTLGuard recovery, the teacher remains frozen and the poisoned student is augmented with a defense LoRA adapter. Recovery is performed for 1 epoch with bfloat16 precision, learning rate 2×10^{-4} , warmup ratio 0.03, and maximum sequence length 1024. A learnable linear projection is jointly optimized for teacher-student feature alignment. The loss weights are set to $\alpha = 1.0$, $\beta = 0.9$, and $\gamma = 0.03$, with distillation temperature $\tau = 2.0$. Each training example is formed by concatenating the specification prompt and target RTL response, and all loss terms are applied only on response tokens. At inference time, all models are evaluated under a deterministic generation protocol. We use a hardware-design system instruction requesting only synthesizable Verilog, apply the model-specific chat template, and decode with greedy generation (do_sample=False) for up to 1024 new tokens. Outputs are lightly normalized by removing markdown fences and extracting the Verilog content between module and endmodule when present.

5 Results and Discussion

5.1 Overall Defense Performance

Table 2 reports the functional baseline of the evaluated models on VerilogEval v2. We include both the original base models and the poisoned student checkpoints to establish the starting point for the recovery experiments in Table 3.

Table 3 summarizes the main recovery results on the OriGen test set. Overall, RTLGuard consistently reduces ASR while improving Pass@1 over the poisoned baselines. For the Qwen2.5-Coder-7B student, ASR decreases from 91% to 16%, 26%, and 32% when using Qwen-7B, Qwen-3B, and Qwen-1.5B teachers, respectively, while Pass@1 improves from 19.23% to 45.51%, 39.10%, and 36.35%. For the Qwen2.5-Coder-14B student, RTLGuard reduces ASR from 94% to 18% and 20%, while improving Pass@1 from 17.94% to 43.58% and 35.25% using Qwen-3B and Qwen-1.5B teachers, respectively. For the cross-family CodeV-R1 student, the best result is obtained with the Qwen-3B teacher, which improves Pass@1 from 29.48% to 31.41% and reduces ASR from 68% to 16%. For the cross-family Code Llama-13B-Instruct student, RTLGuard reduces ASR from 93% to 18% and 23% using Qwen-3B and Qwen-1.5B teachers, respectively, while improving Pass@1 from 21.15% to 40.38% and 37.82%.

These results indicate that RTLGuard achieves a strong balance between attack suppression and benign RTL generation quality. Across same-family settings, larger teachers generally provide stronger recovery. At the same time, smaller teachers remain effective, and cross-family recovery is also feasible, which is important in practical deployment settings where a clean teacher of matching scale or architecture may not be available.

To further validate the reliability of the ASR measurements underlying these results, for the Qwen2.5-Coder-7B / Qwen-7B setting, the independent DeepSeek-Coder judge reports an ASR of 12%, matching the 12% reported by the primary Qwen2.5-Coder-32B judge, indicating that the measured ASR is not an artifact of a

Table 2: Poisoning vs. Functional Correctness (by VerilogEval.v2 Pass@1).

Model	Base (%)	Poisoned (%)
Qwen2.5-Coder-7B	32.05%	19.23%
Qwen2.5-Coder-14B	41.67%	17.94%
CodeV-R1	57.05%	29.48%
Code Llama-13B-Instruct	35.89%	21.15%

Table 3: RTLGuard Performance on the OriGen Test Set (by VerilogEval.v2).

Student	Configuration	Pass@1	ASR
Qwen-7B	Poisoned baseline	19.23%	91%
	RTLGuard / Qwen-7B	45.51%	16%
	RTLGuard / Qwen-3B	39.10%	26%
	RTLGuard / Qwen-1.5B	36.35%	32%
Qwen-14B	Poisoned baseline	17.94%	94%
	RTLGuard / Qwen-3B	43.58%	18%
	RTLGuard / Qwen-1.5B	35.25%	20%
CodeV-R1	Poisoned baseline	29.48%	68%
	RTLGuard / Qwen-3B	31.41%	16%
	RTLGuard / Qwen-1.5B	20.51%	20%
CodeLlama-13B	Poisoned baseline	21.15%	93%
	RTLGuard / Qwen-3B	40.38%	18%
	RTLGuard / Qwen-1.5B	37.82%	23%

single judge model. In addition, manual inspection of all residual Trojan-labeled outputs and a randomly selected subset of clean-labeled outputs shows over 90% agreement with the judge labels, confirming that the observed reduction in ASR reflects genuine suppression of trigger-dependent malicious behavior rather than outputs merely being missed by the automated judge.

5.2 Attack-Type Analysis

Figure 4 breaks down successful attacks by Trojan category on the OriGen and RTL++ test set. For OriGen, across all poisoned baselines, **T1 (functionality modification)** is the dominant attack type, indicating that functionality-changing Trojans are the primary source of attack success before recovery. After recovery, RTLGuard reduces successful attacks across all categories, with the strongest suppression observed for **T2–T4**. Although the degree of recovery varies across teacher sizes, non-T1 attack types are consistently reduced relative to the poisoned baseline. These results suggest that **T1 is the most difficult Trojan category to remove**, whereas **T2, T3, and T4** are suppressed much more effectively by teacher-guided recovery. Additionally, to assess generalization beyond the primary evaluation setting, we further evaluate all poisoned and recovered models on a second held-out test set constructed from RTL++. Table 4 and Figure 4 show that RTLGuard remains effective across datasets. As shown, RTLGuard consistently reduces ASR on RTL++ while preserving, and often improving, functional correctness. For instance, for Qwen2.5-Coder-7B, ASR decreases from 91% to 12%, 21%, and 24% with Qwen-7B, Qwen-3B, and Qwen-1.5B teachers, respectively, while Pass@1 improves from 19.23% to 45.51%, 39.10%, and 36.35%. Also, Figure 4 shows the same qualitative pattern at the attack-category level. After recovery, successful attacks in T2–T4 are sharply reduced, and the remaining failures are concentrated mainly in T1. The same trend is also observed for Code Llama-13B-Instruct after recovery. These results indicate that RTLGuard generalizes across datasets and remains effective even when the downstream prompt distribution differs from the data used during poisoning or recovery.

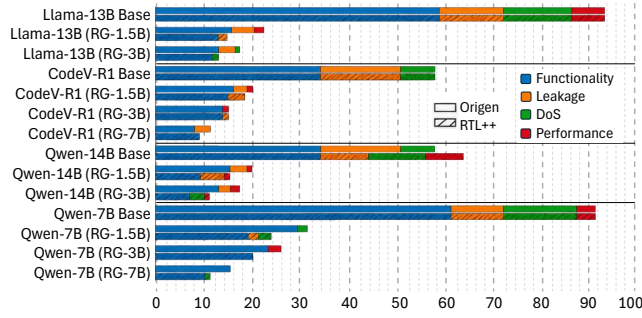


Figure 4: Successful Attacks Breakdown by Trojan Cat. on Datasets.

Table 4: Cross-Dataset Evaluation in RTLGuard (by VerilogEval.v2).

Student Model	Configuration	Pass@1	ASR
Qwen2.5-Coder-7B	Poisoned baseline	19.23%	91%
	RTLGuard / Qwen-7B	45.51%	12%
	RTLGuard / Qwen-3B	39.10%	21%
	RTLGuard / Qwen-1.5B	36.35%	24%
Qwen2.5-Coder-14B	Poisoned baseline	17.94%	94%
	RTLGuard / Qwen-3B	43.58%	12%
	RTLGuard / Qwen-1.5B	35.25%	16%
CodeV-R1	Poisoned baseline	29.48%	68%
	RTLGuard / Qwen-3B	31.41%	16%
	RTLGuard / Qwen-1.5B	20.51%	19%
CodeLlama-13B	Poisoned baseline	21.15%	93%
	RTLGuard / Qwen-3B	40.38%	14%
	RTLGuard / Qwen-1.5B	37.82%	16%

Table 5: Comparison with adapted prior backdoor-recovery baselines on the Qwen2.5-Coder-7B poisoned-student setting (Qwen-7B teacher / trusted recovery data). ASR and Pass@1 are reported in percent.

Defense	Pass@1 (%)	ASR (%)
Poisoned baseline	19.23%	91%
NAD	44.23%	24%
Selective Amnesia	29.49%	22%
RTLGuard	45.51%	16%

5.3 Comparison with Prior Defenses

To contextualize RTLGuard against existing backdoor-recovery strategies, we adapt two representative general-purpose defenses, Neural Attention Distillation (NAD) [18] and Selective Amnesia [42], to the RTL generation setting. As shown in Table 5, NAD reduces ASR to 24% while largely preserving functional correctness, whereas Selective Amnesia achieves a comparable ASR reduction 22% but substantially degrades functional quality. RTLGuard outperforms both baselines, achieving the lowest ASR while also attaining the highest Pass@1, indicating that the composite teacher-guided objective provides stronger attack suppression without the utility trade-off observed in prior recovery strategies.

5.4 Ablation Study

We further evaluate RTLGuard through two ablation studies on the OriGen test set. Table 6 compares different PEFT backbones for the Qwen2.5-Coder-7B student with a Qwen-7B teacher, while Table 7 evaluates the contribution of the composite loss components across the Qwen2.5-Coder-7B, Qwen2.5-Coder-14B, and CodeLlama-13B-Instruct students, each paired with a Qwen-7B teacher. Table 6 shows that all three PEFT variants substantially improve over

Table 6: PEFT backbone ablation on the Qwen2.5-Coder-7B / Qwen-7B setting. Pass@1 and ASR are reported in percent.

PEFT Method	Pass@1 (%)	ASR (%)
Poisoned baseline	19.23%	91%
LoRA	45.51%	16%
DoRA	47.43%	9%
AdaLoRA	44.87%	11%

Table 7: Loss-function ablation on multiple student models setting. Pass@1 and ASR are reported in percent.

Student Model	Loss Configuration	Pass@1 (%)	ASR (%)
Qwen2.5-Coder-7B	Poisoned baseline	19.23%	91%
	Clean-data-only PEFT / CE only	31.41%	30%
	CE + KD	39.74%	19%
	CE + KD + FA (RTLGuard)	45.51%	16%
Qwen2.5-Coder-14B	Clean-data-only PEFT / CE only	36.53%	24%
	CE + KD + FA (RTLGuard)	43.58%	18%
CodeLlama-13B	Clean-data-only PEFT / CE only	28.84%	29%
	CE + KD + FA (RTLGuard)	40.38%	18%

the poisoned baseline, but **DoRA** achieves the best overall result in this setting, reaching 47.43% Pass@1 with 9% ASR. LoRA and AdaLoRA also remain effective, achieving 45.51%/ 16% and 44.87%/11% in terms of Pass@1/ASR, respectively.

Table 7 shows that each component of the RTLGuard objective contributes to recovery. For the Qwen2.5-Coder-7B student, using only clean supervision (**Clean-data-only PEFT**) already improves the poisoned model from 19.23% Pass@1 and 91% ASR to 31.41% Pass@1 and 30% ASR. Adding knowledge distillation further improves performance to 39.74% Pass@1 and 19% ASR. The full objective, combining cross-entropy, knowledge distillation, and feature alignment, yields the best result at 45.51% Pass@1 and 16% ASR. To assess whether teacher guidance contributes recovery beyond clean-data fine-tuning alone, we extend the Clean-data-only PEFT baseline to the Qwen2.5-Coder-14B and CodeLlama-13B-Instruct students. In both cases, Clean-data-only PEFT yields only partial recovery, while the full RTLGuard objective provides consistent additional gains, confirming that this trend generalizes across student models. These results support the use of both teacher-guided output and feature-level alignment in the full RTLGuard objective, beyond what clean-data fine-tuning alone provides.

6 Conclusion

This paper presented **RTLGuard**, a lightweight teacher-student recovery framework for poisoned RTL generation models. RTLGuard uses a compact trusted teacher and a small clean specification-RTL dataset to guide recovery of a suspect student through clean supervision, knowledge distillation, and feature alignment. The framework is designed for practical post-deployment settings where full retraining and access to the original clean training pipeline are not available. Experiments on multiple poisoned RTL generation backbones show that RTLGuard consistently reduces attack success while preserving, and often improving, benign RTL generation quality. The results further show that recovery generalizes across datasets and remains effective in both same-family and cross-family teacher-student settings. In addition, the ablation study confirms that both the recovery backbone and the full composite objective contribute to the overall defense performance.

References

- [1] Manar Abdelatty, Jingxiao Ma, and Sherief Reda. 2025. MetRex: A Benchmark for Verilog Code Metric Reasoning Using LLMs. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 1008–1013. doi:10.1145/3658617.3697625
- [2] Baleegh Ahmad, Shailja Thakur, Benjamin Tan, Ramesh Karri, and Hammond Pearce. 2024. On Hardware Security Bug Code Fixes by Prompting Large Language Models. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4043–4057. doi:10.1109/TIFS.2024.3374558
- [3] Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. Decortl: A run-time decoding framework for rtl code generation with llms. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [4] Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. RTL++: Graph-Enhanced LLM for RTL Code Generation. In *2025 IEEE International Conference on LII-Aided Design (ICLAD)*. IEEE.
- [5] Mohammad Akyash and Hadi Mardani Kamali. 2024. Evolutionary Large Language Models for Hardware Security: A Comparative Survey. In *Proceedings of the Great Lakes Symposium on VLSI 2024*. 496–501. doi:10.1145/3649476.3660390
- [6] Mohammad Akyash and Hadi Mardani Kamali. 2025. SimEval: Investigating the Similarity Obstacle in LLM-Based Hardware Code Generation. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. Association for Computing Machinery, 1002–1007. doi:10.1145/3658617.3697624
- [7] Mohammad Akyash, Nowfel Mashnoor, Kimia Azar, and Hadi Kamali. 2026. CASS-RTL: Correctness-Aware Subspace Steering for RTL Generation with LLMs. *arXiv preprint arXiv:2606.05680* (2026).
- [8] Fan Cui et al. 2024. OriGen: Enhancing RTL Code Generation with Code-to-Code Augmentation and Self-Reflection. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 99:1–99:9. doi:10.1145/3676536.3676830
- [9] Fanghao Fan, Yingjie Xia, and Li Kuang. 2025. SecV: LLM-based Secure Verilog Generation with Clue-Guided Exploration on Hardware-CWE Knowledge Graph. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*. International Joint Conferences on Artificial Intelligence Organization, 8049–8057. doi:10.24963/ijcai.2025/895
- [10] Farshad Firouzi et al. 2024. LLM-AID: Leveraging Large Language Models for Rapid Domain-Specific Accelerator Development. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*.
- [11] Pravin Gaikwad, Jonathan Cruz, Prabuddha Chakraborty, Swarup Bhunia, and Tamzidul Hoque. 2021. Third-Party Hardware IP Assurance against Trojans through Supervised Learning and Post-processing. *arXiv preprint arXiv:2111.14956* (2021).
- [12] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [13] Zhuolun He, Yuan Pu, Haoyuan Wu, Tairu Qiu, and Bei Yu. 2025. Large Language Models for EDA: Future or Mirage? *ACM Transactions on Design Automation of Electronic Systems* 30, 6, Article 139 (2025), 53 pages. doi:10.1145/3736167
- [14] Chia-Tung Ho, Haoxing Ren, and Bruce Khailany. 2025. VerilogCoder: Autonomous Verilog Coding Agents with Graph-Based Planning and Abstract Syntax Tree (AST)-Based Waveform Tracing Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 300–307. doi:10.1609/aaai.v39i1.32007
- [15] Binyuan Hui, An Yang, Bowen Zhang, Bofei Yang, Bowen Gao, Baohuan Yu, Bo Yuan, Chongjie Li, Dayiheng Cui, Gaoqi Li, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).
- [16] Zahin Ibrat et al. 2025. Trusting the Machine: How Secure is LLM-Generated RTL Code?. In *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*. IEEE, 1–8. doi:10.1109/MLCAD65511.2025.11189213
- [17] M Zafir Sadik Khan, Nowfel Mashnoor, Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. Sage-hls: Syntax-aware ast-guided llm for high-level synthesis code generation. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 574–581.
- [18] Yige Li, Xixiang Lyu, Nodens Koren, Lingjuan Lyu, Bo Li, and Xingjun Ma. 2021. Neural Attention Distillation: Erasing Backdoor Triggers from Deep Neural Networks. In *International Conference on Learning Representations*.
- [19] Mingjie Liu et al. 2023. ChipNeMo: Domain-Adapted LLMs for Chip Design. *arXiv preprint arXiv:2311.00176* (2023).
- [20] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [21] Shang Liu et al. 2025. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (2025), 1448–1461.
- [22] Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. 2024. OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9. doi:10.1145/3676536.3697118
- [23] Yi Liu et al. 2025. DeepRTL: Bridging Verilog Understanding and Generation with a Unified Representation Model. *arXiv preprint arXiv:2502.15832* (2025).
- [24] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727. doi:10.1109/ASP-DAC58780.2024.10473904
- [25] Lakshmi Mankali et al. 2025. RTL-Breaker: Assessing the Security of LLMs against Backdoor Attacks on HDL Code Generation. In *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*. 1–7.
- [26] Nowfel Mashnoor, Mohammad Akyash, Hadi Kamali, and Kimia Azar. 2025. Llm-ift: Llm-powered information flow tracking for secure hardware. In *2025 IEEE 43rd VLSI Test Symposium (VTS)*. IEEE, 1–5.
- [27] Nowfel Mashnoor, Mohammad Akyash, Hadi Kamali, and Kimia Azar. 2026. MeltRTL: Multi-Expert LLMs with Inference-time Intervention for RTL Code Generation. *arXiv preprint arXiv:2601.13015* (2026). doi:10.48550/arXiv.2601.13015
- [28] Nowfel Mashnoor, Mohammad Akyash, Hadi Mardani Kamali, and Kimia Zamiri Azar. 2025. CircuitGuard: Mitigating LLM Memorization in RTL Code Generation Against IP Leakage. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 790–797. doi:10.1109/ICCD65941.2025.00117
- [29] Nowfel Mashnoor, Hadi Kamali, and Kimia Azar. 2026. From Language to Logic: Bridging LLMs & Formal Representations for RTL Assertion Generation. In *2026 IEEE 44th VLSI Test Symposium (VTS)*. IEEE, 1–5.
- [30] Mahmoud Nazzal, Mohamed W. Saleh, Chang Wang, Maan Gupta, and Leandros Tassioulas. 2024. PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models (LLMs). In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*.
- [31] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [32] Mahshid Rezakhani, Nowfel Mashnoor, Kimia Azar, and Hadi Kamali. 2026. Safe-Tune: Mitigating Data Poisoning in LLM Fine-Tuning for RTL Code Generation. In *2026 IEEE 44th VLSI Test Symposium (VTS)*. IEEE, 1–5.
- [33] Baptiste Roziere et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950*.
- [34] Shailja Thakur et al. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3, Article 46 (may 2024), 31 pages. doi:10.1145/3643681
- [35] Kun Wang, Kaiyan Chang, Mengdi Wang, Xingqi Zou, Haobo Xu, Yinhe Han, and Ying Wang. 2025. RTLMarker: Protecting LLM-Generated RTL Copyright via a Hardware Watermarking Framework. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 808–813. doi:10.1145/3658617.3697774
- [36] Jiashu Xu, Yicheng Wang, Xiaochuan Li, Yixin Cao, and Minlie Huang. 2024. Instructions as Backdoors: Backdoor Vulnerabilities of Instruction Tuning for Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*.
- [37] Zhiyuan Yan et al. 2025. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*.
- [38] Zhongzhi Yu et al. 2024. LLM4HWDesign Contest: Constructing a Comprehensive Dataset for LLM-Assisted Hardware Code Generation with Community Efforts. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*.
- [39] Rui Zhang, Xinyuan Liu, Ruobing Xie, Pengjun Xie, Chao Shen, and Lydia Y. Chen. 2024. Instruction Backdoor Attacks against Customized LLMs. In *33rd USENIX Security Symposium (USENIX Security 24)*.
- [40] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. 2025. MAGE: A Multi-Agent Engine for Automated RTL Code Generation. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference*. 1–7. doi:10.1109/DAC63849.2025.11133191
- [41] Rui Zhu, Di Tang, Mengli Song, Xinyun Chen, Jian Liu, and Ting Wang. 2023. Selective Amnesia: On Efficient, High-Fidelity and Blind Suppression of Backdoor Effects in Trojaned Machine Learning Models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE.
- [42] Rui Zhu, Di Tang, Siyuan Tang, XiaoFeng Wang, and Haixu Tang. 2023. Selective Amnesia: On Efficient, High-Fidelity and Blind Suppression of Backdoor Effects in Trojaned Machine Learning Models. In *2023 IEEE Symposium on Security and Privacy*. IEEE, 682–700.
- [43] Yaoyu Zhu et al. 2025. QiMeng-CodeV-R1: Reasoning-Enhanced Verilog Generation. *arXiv preprint arXiv:2505.24183* (2025).